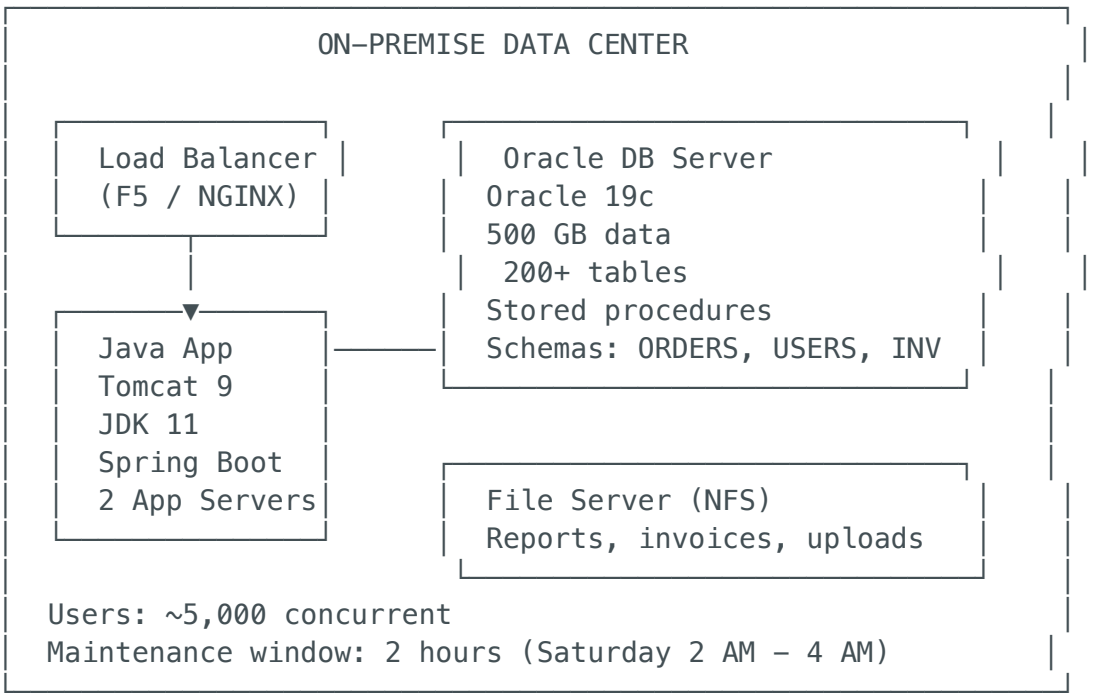
Project 2: Legacy Java Application Migration to AWS — PoC Architecture

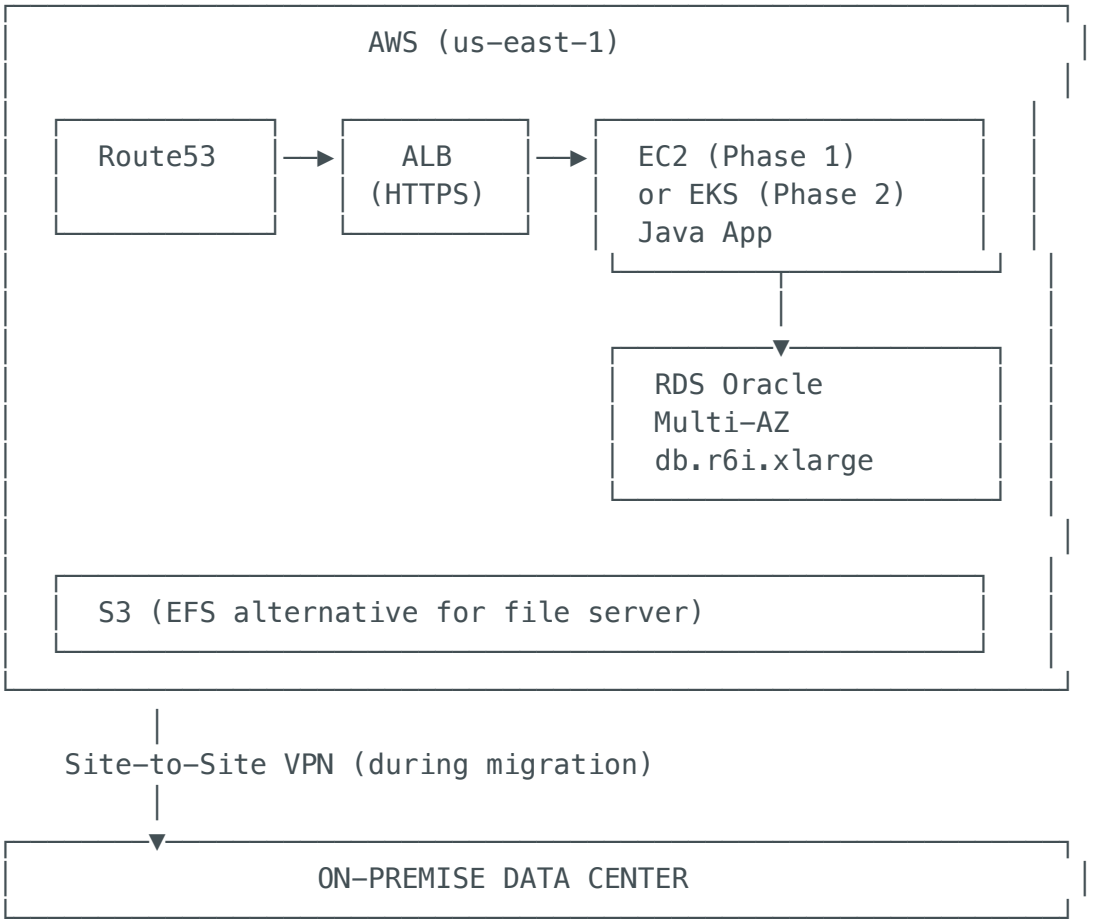
> **Migration:** On-Premise Java App → AWS EC2/EKS using **Application Migration Service (MGN)**
> **Database:** Oracle on-prem → Amazon RDS Oracle using **AWS DMS** (CDC)
> **Connectivity:** Site-to-Site VPN
> **Goal:** Zero/minimal downtime cutover to production

---

1. Current State (On-Premise)



2. Target State (AWS)



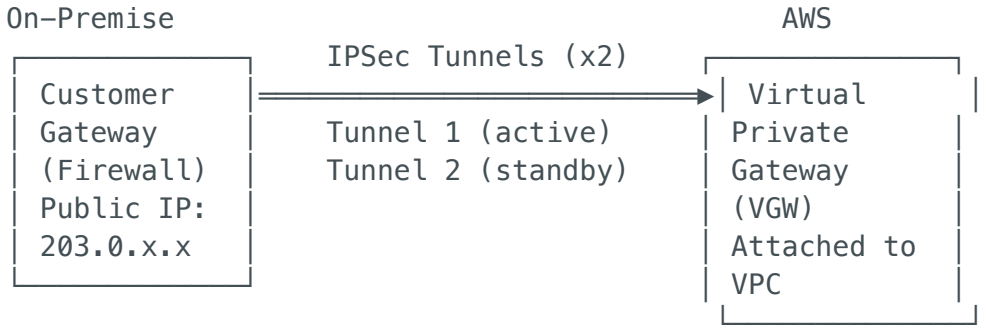
3. Migration Phases — End-to-End Flow

Timeline Overview

```
Week 1–2     Week 3–4     Week 5–6     Week 7       Week 8
──────────────────────────────────────────────────────────────
PREPARE      REPLICATE    TEST         CUTOVER      DECOMMISSION
  │              │           │            │            │
VPN Setup    MGN Agent    Test in AWS  DNS Switch   Power off
DMS Setup    DMS Full     UAT          Verify       on-prem
Landing Zone Load+CDC     Load Test    Monitor
```

Phase 1: Preparation (Week 1-2)

Step 1: Establish Site-to-Site VPN




```
Routing:
  On-prem 192.168.0.0/16 ↔ AWS VPC 10.0.0.0/16
  BGP or static routes propagated to VPC route tables
```

VPN Configuration (Terraform):

```
resource "aws_vpn_gateway" "main" {
  vpc_id = aws_vpc.main.id
  tags   = { Name = "migration-vpg" }
}

resource "aws_customer_gateway" "onprem" {
  bgp_asn    = 65000
  ip_address = "203.0.113.100"  # On-prem firewall public IP
  type       = "ipsec.1"
```

```
  tags       = { Name = "onprem-cgw" }
}

resource "aws_vpn_connection" "main" {
  vpn_gateway_id      = aws_vpn_gateway.main.id
  customer_gateway_id = aws_customer_gateway.onprem.id
  type                = "ipsec.1"
  static_routes_only  = false  # Use BGP
  tags                = { Name = "onprem-to-aws-vpn" }
}
```

Step 2: Set Up AWS Landing Zone

```
VPC (10.0.0.0/16)
├── Public Subnets (3 AZs)
│   ├── 10.0.1.0/24 — ALB, NAT Gateway
│   ├── 10.0.2.0/24 — ALB
│   └── 10.0.3.0/24 — ALB
├── Private App Subnets (3 AZs)
│   ├── 10.0.101.0/24 — EC2 / EKS nodes
│   ├── 10.0.102.0/24 — EC2 / EKS nodes
│   └── 10.0.103.0/24 — EC2 / EKS nodes
├── Private DB Subnets (3 AZs)
│   ├── 10.0.201.0/24 — RDS Primary
│   ├── 10.0.202.0/24 — RDS Standby
│   └── 10.0.203.0/24 — (future)
└── Security Groups
    ├── sg-alb: 443 from 0.0.0.0/0
    ├── sg-app: 8080 from sg-alb
    ├── sg-db:  1521 from sg-app
    └── sg-mgn: 443,1500 from on-prem CIDR
```

Step 3: Set Up AWS DMS for Database Migration

```
┌──────────────────────────────────────────────────────────────┐
│                    AWS DMS Architecture                      │
│                                                              │
│  On-Prem Oracle ──(VPN)──► DMS Replication Instance ──► RDS Oracle│
│  (Source)                    (dms.r5.xlarge)              (Target) │
│                                                              │
│  Endpoints:                                                  │
│    Source: oracle://192.168.1.50:1521/PRODDB                 │
│    Target: oracle://rds-endpoint.us-east-1.rds.amazonaws.com:1521│
│                                                              │
│  Replication Task:                                           │
```

```
│    Type: Full Load + CDC (Change Data Capture)                    │
│    Tables: All (200+)                                             │
│    LOB mode: Limited (for performance)                            │
│    Parallel load: 8 threads                                       │
│    CDC start: After full load completes                           │
└──────────────────────────────────────────────────────────────────┘
```

DMS Task Flow:

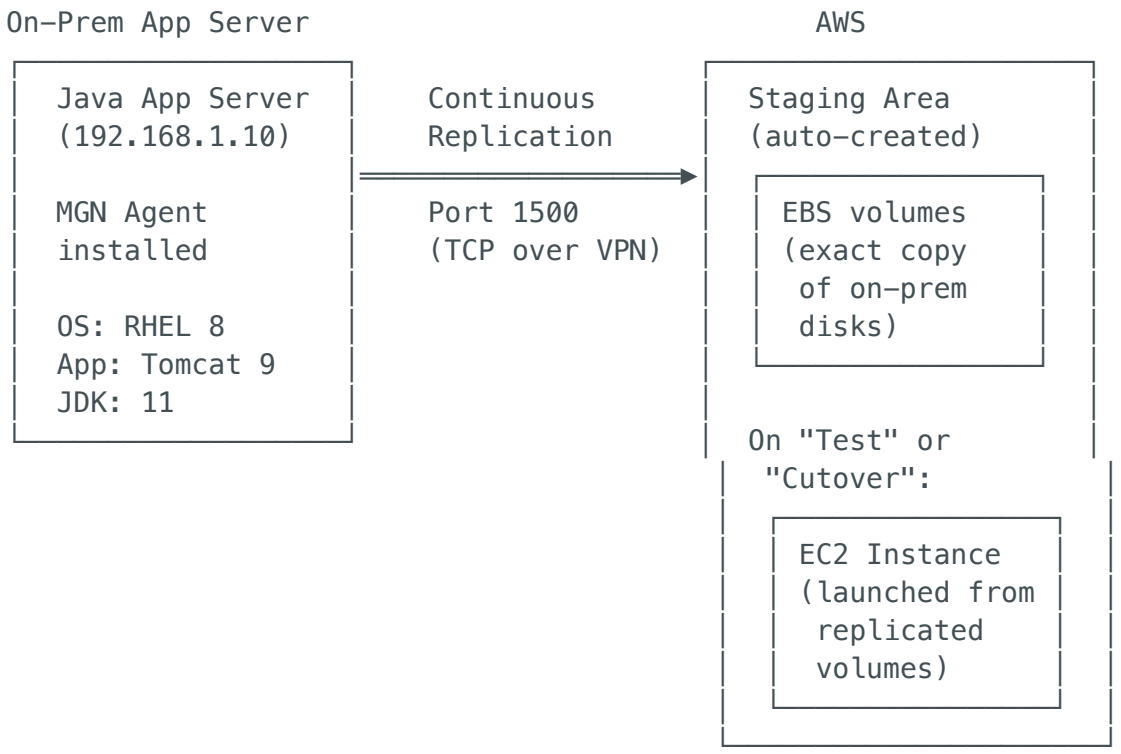
```
Step 1: Create RDS Oracle instance (Multi-AZ)
          ├─ Same Oracle version (19c)
          ├─ Parameter group matching on-prem settings
          └─ Option group with required Oracle options

Step 2: Create DMS Replication Instance
          ├─ dms.r5.xlarge (enough for 500GB)
          ├─ Multi-AZ: Yes
          └─ VPC: Same as RDS, with route to on-prem via VPN

Step 3: Create Source Endpoint (On-Prem Oracle)
          ├─ Connection via VPN
          ├─ Supplemental logging enabled on source
          └─ Test connection: PASS

Step 4: Create Target Endpoint (RDS Oracle)
          ├─ Direct connection within VPC
          └─ Test connection: PASS

Step 5: Create Replication Task
          ├─ Migration type: "Migrate existing data + replicate ongoing
changes"
          ├─ Table mappings: Include all schemas (ORDERS, USERS, INV)
          ├─ Transformation rules: (if schema rename needed)
          ├─ Enable validation: Yes
          ├─ Enable CloudWatch logs: Yes
          └─ Start task: Immediately
```

DMS Migration Timeline:

```
Hour 0-4:     Full Load (500GB over VPN)
                ├─ 8 parallel threads
                ├─ ~125 MB/s over VPN
                └─ Tables loaded in dependency order (FK constraints)

Hour 4+:      CDC (Change Data Capture) begins
                ├─ Reads Oracle redo logs
                ├─ Replicates INSERT/UPDATE/DELETE in near-real-time
                ├─ Latency: 1-5 seconds behind source
                └─ Continues until cutover
```

Phase 2: Application Replication with MGN (Week 3-4)

AWS Application Migration Service (MGN) Flow



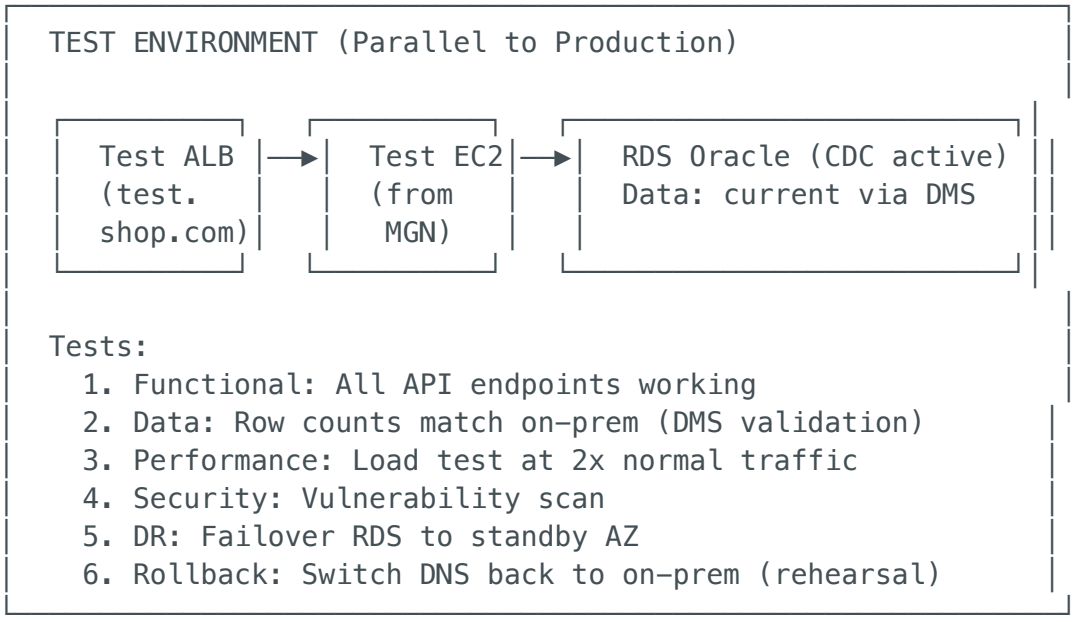
MGN Step-by-Step:

```
Step 1: Initialize MGN in AWS Console
          ├─ Set replication server template (subnet, SG, instance type)
          └─ Configure staging area (subnet for replication servers)

Step 2: Install MGN Agent on source server (via VPN/SSH)
          $ wget -O ./aws-replication-installer-init \
            https://aws-application-migration-service-us-east-
1.s3.amazonaws.com/latest/linux/aws-replication-installer-init
          $ sudo python3 aws-replication-installer-init \
            --region us-east-1 \
            --aws-access-key-id AKIA... \
            --aws-secret-access-key ...
          # Agent begins continuous block-level replication

Step 3: Wait for initial sync to complete
          ├─ Progress visible in MGN console
```

```
           ├─ 500GB disk → ~2-4 hours over VPN
           └─ Status changes: "Initial sync" → "Healthy" (ready for test)

Step 4: Configure Launch Template
           ├─ Instance type: m5.xlarge (matching on-prem specs)
           ├─ Subnet: Private app subnet
           ├─ Security group: sg-app
           ├─ IAM role: EC2 role for CloudWatch, SSM
           └─ Post-launch script: Update JDBC URL to RDS endpoint

Step 5: Test Launch
           ├─ MGN creates EC2 from replicated volumes
           ├─ EC2 boots with exact same OS, app, configs
           ├─ Update application.properties → point to RDS
           ├─ Verify app starts, connects to RDS
           ├─ Run smoke tests
           └─ Terminate test instance (replication continues)
```

Phase 3: Testing & Validation (Week 5-6)

Test Plan



DMS Validation Queries

```sql
-- Row count comparison (run on both source and target)
SELECT table_name, num_rows FROM all_tables WHERE owner = 'ORDERS';
```

```sql
-- DMS table statistics
SELECT * FROM awsdms_control.awsdms_validation_failures_v1;

-- Check CDC lag
-- AWS Console → DMS → Replication Task → Statistics
-- CDCLatencySource: should be < 5 seconds
-- CDCLatencyTarget: should be < 10 seconds
```

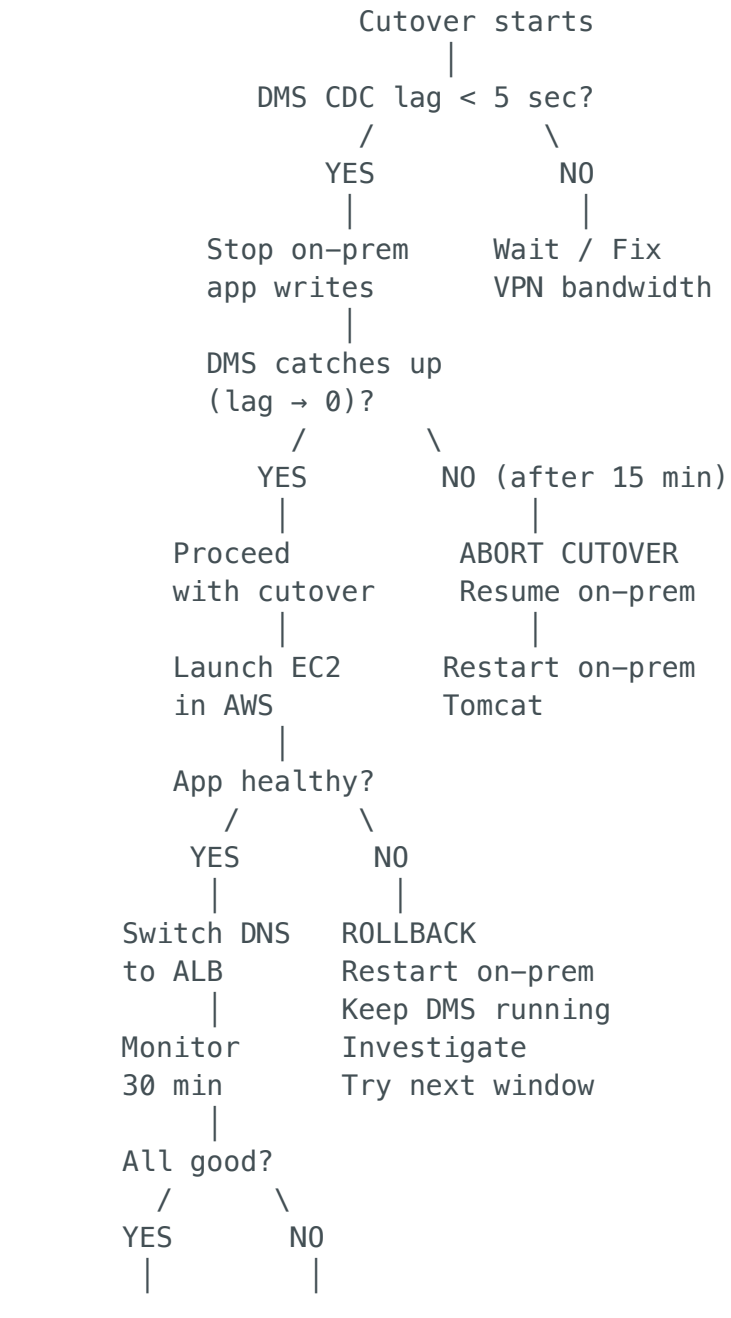
Phase 4: Production Cutover (Week 7)

Cutover Plan — Zero/Minimal Downtime

```
TIME            ACTION                                    DOWNTIME?
──────────────────────────────────────────────────────────────────
T-24h           Final load test on AWS                     No
T-12h           Notify users of maintenance window         No
T-4h            Verify DMS CDC lag < 2 seconds             No
T-2h            Reduce DNS TTL to 60 seconds               No

T-0 (2:00 AM)   ═══ CUTOVER BEGINS ═══                    Starts

T+0 min         Stop application writes on-prem            YES (start)
                (Set app to read-only mode OR
                 stop Tomcat on on-prem)

T+2 min         Wait for DMS CDC to catch up                YES
                (CDCLatencyTarget → 0 seconds)
                Verify: no pending changes

T+5 min         Stop DMS Replication Task                   YES
                (Status: Stopped)

T+7 min         Run data validation queries                 YES
                Compare row counts (source vs target)
                Verify critical tables match

T+10 min        Launch MGN Cutover Instance                 YES
                (or start EC2 if already launched)
                Update application.properties:
                  DB URL → RDS endpoint
                  File storage → S3/EFS

T+15 min        Verify EC2 app starts successfully          YES
                Health check: curl http://EC2:8080/health
                Smoke test: Create test order

T+20 min        Update Route53 DNS                          YES
                shop.com → ALB (AWS)
```

```
               (was pointing to on-prem LB)

T+25 min       DNS propagation + verification        YES (ending)
               Monitor: Real user traffic flowing
               Check: No errors in CloudWatch

T+30 min       ═══ CUTOVER COMPLETE ═══             No (restored)

T+1h           Monitor error rates, latency
T+4h           Confirm no rollback needed
T+24h          Increase DNS TTL back to 300s
T+7d           Decommission on-prem (Phase 5)

TOTAL DOWNTIME: ~25-30 minutes
```

Cutover Decision Tree



```
    DONE ✅      ROLLBACK
                 DNS → on-prem
```

Phase 5: Post-Migration & Decommission (Week 8)

```
Step 1: Monitor AWS production (7 days)
          ├─ CloudWatch dashboards: CPU, memory, error rates
          ├─ RDS Performance Insights: Query performance
          ├─ Compare response times: AWS vs on-prem baseline
          └─ User feedback

Step 2: Optimize AWS resources
          ├─ Right-size EC2 based on actual usage
          ├─ Enable RDS Reserved Instance (if committing)
          ├─ Set up automated backups and snapshots
          └─ Configure CloudWatch alarms

Step 3: Decommission on-prem
          ├─ Keep on-prem running (read-only) for 30 days as safety net
          ├─ Remove VPN connection (or keep for other workloads)
          ├─ Delete DMS replication task and instance
          └─ Power off on-prem servers after 30-day soak period

Step 4: Phase 2 modernization (optional)
          ├─ Containerize Java app → Docker → EKS
          ├─ Implement CI/CD pipeline
          └─ Consider Aurora migration for cost/performance
```

---

5. End-to-End Flow After Migration

```
User (Browser/Mobile)
      │
      ▼
Route53 (shop.com → ALB)
      │
      ▼
ALB (HTTPS:443, ACM cert)
      ├─ /api/* → Target Group: App EC2 (port 8080)
      └─ /* → Target Group: App EC2 (port 8080)
      │
      ▼
```

```
EC2 (m5.xlarge, Private Subnet)
      │  Java 11 + Spring Boot + Tomcat
      │  Migrated via MGN (exact copy of on-prem)
      │
      ├──► RDS Oracle (Writer) — Port 1521
      │    Orders, Users, Inventory writes
      │
      ├──► RDS Oracle (Reader) — Port 1521
      │    Catalog reads, report queries
      │
      ├──► S3 (file storage — replaced NFS)
      │    Invoices, reports, uploads
      │
      └──► ElastiCache Redis (session store)
           Replaces sticky sessions for HA
```

---

6. Rollback Plan

Scenario	Rollback Action	Time
App fails on AWS EC2	DNS → on-prem LB, restart on-prem Tomcat	5 min
RDS data issues	DMS reverse replication (AWS → on-prem)	30 min
VPN drops during cutover	Abort, resume on-prem	2 min
Performance degradation	Scale up EC2, optimize RDS, or rollback DNS	10-30 min

Key: DMS CDC runs continuously until decommission. On-prem stays warm for 30 days. Rollback is always available.

---

7. Key Services & Their Role

AWS Service	Role in Migration
Site-to-Site VPN	Encrypted connectivity between on-prem and AWS
MGN	Replicates on-prem servers → EC2 (block-level, continuous)
DMS	Migrates Oracle DB with full load + CDC (near-zero downtime)
RDS Oracle	Target database (Multi-AZ, automated backups, encryption)

AWS Service	Role in Migration
ALB	HTTPS load balancing with health checks
Route53	DNS cutover (on-prem → AWS)
CloudWatch	Monitoring, alerting, dashboards
S3	Replace on-prem NFS for file storage
ACM	TLS certificates for ALB
Systems Manager	Patch management, remote access (replaces SSH)

8. Risk Mitigation

Risk	Mitigation
VPN bandwidth insufficient for full load	Compress DMS data, schedule during off-hours, consider Snowball for initial load
Oracle compatibility issues on RDS	Use SCT (Schema Conversion Tool) to assess before migration
Stored procedures not supported	RDS Oracle supports PL/SQL natively — test all procedures in Phase 3
Application hardcoded IPs	Search/replace in config files during MGN post-launch script
CDC lag during cutover	Pre-cutover rehearsal, ensure VPN bandwidth reserved
DNS propagation delay	Reduce TTL to 60s 24 hours before cutover